

骰子模擬器 Java 程式實作

<https://gemini.google.com/app/b96eed06be7516de>

User prompt: 檔案名稱Exercise 1，用Java寫 題目:實作一個「骰子模擬器」。規格: ●視窗標題「骰子模擬器」,尺寸400×320,關閉時結束程式,開啟時 置中 ● 中央有一個JLabel顯示目前點數,字體60pt ● 下方一個「擲骰子」按鈕 ● 每按一次隨機產生1-6的點數並更新顯示 ● 視窗上方顯示「已擲N次,總和M,平均X.XX」 ● 點數為6時文字變綠色,為1時變紅色,其餘黑色

Response: Exercise1.java 的完整 Java Swing 程式碼如下：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class Exercise1 extends JFrame {
    private JLabel countLabel;
    private JLabel diceLabel;
    private JButton rollButton;

    private int totalCount = 0;
    private int totalSum = 0;
    private Random random = new Random();

    public Exercise1() {
        // 設定視窗標題與尺寸
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // 開啟時置中
        setLayout(new BorderLayout());

        // 上方：顯示統計資訊
        countLabel = new JLabel("已擲0次,總和0,平均0.00", SwingConstants.CENTER);
        countLabel.setFont(new Font("SansSerif", Font.PLAIN, 14));
        add(countLabel, BorderLayout.NORTH);

        // 中央：顯示目前點數
        diceLabel = new JLabel("-", SwingConstants.CENTER);
        diceLabel.setFont(new Font("SansSerif", Font.BOLD, 60));
        diceLabel.setForeground(Color.BLACK);
        add(diceLabel, BorderLayout.CENTER);

        // 下方：擲骰子按鈕
        rollButton = new JButton("擲骰子");
        rollButton.setFont(new Font("SansSerif", Font.PLAIN, 16));
        add(rollButton, BorderLayout.SOUTH);

        // 按鈕事件處理
        rollButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // 隨機產生 1-6 的點數
                int point = random.nextInt(6) + 1;

                // 更新統計數據
                totalCount++;
                totalSum += point;
                double average = (double) totalSum / totalCount;

                // 更新上方資訊 (平均值保留兩位小數)
                countLabel.setText(String.format("已擲%d次,總和%d,平均%.2f", totalCount, totalSum, average));

                // 更新中央點數
                diceLabel.setText(String.valueOf(point));

                // 依據點數改變顏色
                if (point == 6) {

```

```
        diceLabel.setForeground(Color.GREEN);
    } else if (point == 1) {
        diceLabel.setForeground(Color.RED);
    } else {
        diceLabel.setForeground(Color.BLACK);
    }
    }
    });
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new Exercise1().setVisible(true);
        }
    });
}
```

規格落實說明

1. **檔名與主類別**：檔名為 Exercise1.java，類別名稱同為 Exercise1。
2. **視窗規格**：標題設為「骰子模擬器」，尺寸為 400 × 320，setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) 確保關閉時結束程式，setLocationRelativeTo(null) 實現視窗開啟時置中。
3. **介面佈局**：
 - 上方 NORTH：顯示統計狀態（已擲 N 次，總和 M ，平均 $X.XX$ ）。
 - 中央 CENTER：60pt 大字體的 JLabel 顯示當前點數。
 - 下方 SOUTH：「擲骰子」按鈕。
4. **邏輯與顏色規則**：點數為 1 時文字為紅色 (Color.RED)、6 時為綠色 (Color.GREEN)，其餘點數保持黑色 (Color.BLACK)。

User prompt: 解釋一下程式碼

Response: 這段 Java Swing 程式碼主要由三個部分組成：**物件屬性（變數）、介面與邏輯建構子、以及程式執行入口（main 方法）**。以下為完整的分區拆解說明：

1. 類別宣告與成員變數 (Fields)

```
public class Exercise1 extends JFrame {
    private JLabel countLabel; // 上方的統計文字標籤
    private JLabel diceLabel;  // 中央的點數標籤
    private JButton rollButton; // 下方的擲骰子按鈕

    private int totalCount = 0; // 累計擲骰子次數
    private int totalSum = 0;   // 累計點數總和
    private Random random = new Random(); // 亂數產生器
```

- **extends JFrame**：讓 Exercise1 具備 Swing 視窗的基本功能。
- **JLabel 與 JButton**：宣告 UI 元件，在全域宣告方便後續在按鈕事件中讀取與修改。
- **統計變數**：totalCount 與 totalSum 用來記錄歷史紀錄，以計算平均值。

2. 建構子與 UI 佈局 (Constructor & Layout)

(1) 視窗基礎設定

```
setTitle("骰子模擬器");
setSize(400, 320);
```

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 關閉視窗時結束程式
setLocationRelativeTo(null);                      // 讓視窗顯示於螢幕正中央
setLayout(new BorderLayout());                     // 使用方位佈局 (NORTH, CENTER, SOUTH)
```

(2) 元件初始化與排版

```
// 上方：統計標籤
countLabel = new JLabel("已擲0次,總和0,平均0.00", SwingConstants.CENTER);
countLabel.setFont(new Font("SansSerif", Font.PLAIN, 14));
add(countLabel, BorderLayout.NORTH);

// 中央：點數標籤 (初始顯示 "-")
diceLabel = new JLabel("-", SwingConstants.CENTER);
diceLabel.setFont(new Font("SansSerif", Font.BOLD, 60)); // 60pt 大字體
diceLabel.setForeground(Color.BLACK);
add(diceLabel, BorderLayout.CENTER);

// 下方：按鈕
rollButton = new JButton("擲骰子");
rollButton.setFont(new Font("SansSerif", Font.PLAIN, 16));
add(rollButton, BorderLayout.SOUTH);
```

- **BorderLayout 方位配置**：將標籤與按鈕分別放入 NORTH（北）、CENTER（中）、SOUTH（南），符合題目對版面的要求。

3. 事件處理與核心邏輯 (Event Handling)

每當使用者點擊「擲骰子」按鈕時，會觸發 actionPerformed 方法：

```
rollButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // 1. 隨機產生 1~6 的整數
        int point = random.nextInt(6) + 1;

        // 2. 更新統計數據
        totalCount++;
        totalSum += point;
        double average = (double) totalSum / totalCount; // 強制轉型為 double 確保計算精確度

        // 3. 更新上方統計文字 (格式化保留兩位小數)
        countLabel.setText(String.format("已擲%d次,總和%d,平均%.2f", totalCount, totalSum, average));

        // 4. 更新中央點數顯示
        diceLabel.setText(String.valueOf(point));

        // 5. 點數顏色判斷
        if (point == 6) {
            diceLabel.setForeground(Color.GREEN); // 6 變綠色
        } else if (point == 1) {
            diceLabel.setForeground(Color.RED); // 1 變紅色
        } else {
            diceLabel.setForeground(Color.BLACK); // 其餘變黑色
        }
    }
});
```

- **random.nextInt(6) + 1**：nextInt(6) 會產生 0 ~ 5 的數字，加上 1 後即轉為 1 ~ 6。
- **(double) totalSum / totalCount**：若不加 (double)，Java 會進行整數除法而捨去小數點。

4. 程式啟動點 (Main Method)

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
```

```
        new Exercise1().setVisible(true); // 實例化並顯示視窗
    }
});
}
```

- **SwingUtilities.invokeLater**：這是 Swing 的標準最佳實踐（Best Practice）。它確保 GUI 元件的建立與更新都在專門處理 UI 的執行緒（Event Dispatch Thread）上執行，避免多執行緒競爭造成的畫面閃爍或錯誤。
-